
PROJECT: To-Do List (Full-Stack CRUD App)

GOAL

A small project where a user can

-  add tasks,
-  view tasks,
-  edit tasks, and
-  delete tasks.

FLOW OVERVIEW

Frontend (React / HTML + JS)



Backend (Node.js + Express)



Database (MongoDB / JSON / MySQL)

FRONTEND FLOW

Main Features

Operation	Feature	Description
 Create	Add Task	Input box + “Add” button to create a new to-do item.
 Read	View Tasks	Display list of all tasks (with checkbox/status).
 Update	Edit / Mark as Done	Edit task text or mark it completed.
 Delete	Remove Task	Delete button to remove a task from the list.

Typical Components (React Example)

1. **App.jsx** – Main container
2. **TodoInput.jsx** – Text input + “Add” button
3. **TodoList.jsx** – Displays all tasks
4. **TodoItem.jsx** – Single task (with Edit/Delete options)

Frontend Logic

- When user adds a task → send POST request to backend.

- Fetch tasks from backend on page load (GET).
- When user edits or deletes → send PUT or DELETE request.
- UI automatically updates after each operation.

BACKEND FLOW (Node.js + Express)

CRUD API Endpoints

Method	Endpoint	Purpose
POST	/tasks	Create a new task
GET	/tasks	Get all tasks
PUT	/tasks/:id	Update a task (text/status)
DELETE	/tasks/:id	Delete a task

Middleware & Logic

- Use `express.json()` to parse request data.
- Each route handles its own CRUD operation.
- Validate inputs (no empty task names).

DATABASE FLOW

You can use **any** of these:

- **JSON file** – for beginners (local storage).
- **MongoDB** – for real-world app.
- **MySQL** – relational version.

Example MongoDB Schema

```
{
  _id: ObjectId,
  title: "Buy groceries",
  isCompleted: false,
  createdAt: "2025-10-08T10:00:00Z"
}
```

COMPLETE PROJECT FLOW

1. User opens frontend → sees list of tasks.
 2. User types a task → clicks “Add.”
 - Frontend sends POST /tasks to backend.
 - Backend saves to DB and returns updated list.
 3. Frontend displays all tasks (GET /tasks).
 4. User clicks “Edit” or “Mark Done.”
 - Frontend sends PUT /tasks/:id.
 - Backend updates DB and returns success.
 5. User clicks “Delete.”
 - Frontend sends DELETE /tasks/:id.
 - Backend removes task from DB.
 6. Page updates instantly using state re-rendering.
-

EXTRA OPTIONAL FEATURES

- Filter: show “All / Completed / Pending” tasks.
 - Search bar: filter tasks by keyword.
 - Local Storage backup (so tasks don’t vanish on refresh).
 - Responsive design (mobile-friendly).
 - Dark/light theme toggle.
-

END RESULT

- Simple, beginner-friendly CRUD project.
- Core understanding of full-stack data flow.
- Ready to extend later (e.g., add user login, categories, due dates).